

PhysioBand AI

Intelligent Resistance Exercise Quality Monitoring System

Complete Hardware and Software Setup Guide

ESP32 firmware - React Native app - FastAPI backend - Python ML pipeline - MongoDB Atlas

IMPORTANT: PhysioBand AI provides exercise feedback for training purposes only. It is not a medical device, does not provide clinical diagnosis and does not guarantee injury prevention. The included AI model is a DEMO model trained on synthetic data and is not validated.

Contents

Chapter	Title
1	Project overview
2	Components required
3	Hardware assembly
4	Circuit connections
5	Development environment setup
6	ESP32 firmware setup
7	Frontend (mobile app) setup
8	Backend setup
9	MongoDB Atlas setup
10	ML model setup
11	Connecting everything
12	Testing and troubleshooting
13	Complete project execution (demonstration day)

Chapter 1 - Project Overview

1.1 Objective

PhysioBand AI is a wearable, AI-assisted resistance-band exercise monitor. It measures **upper-arm movement**, **forearm movement** and **resistance-band tension**, then recognises the exercise, counts repetitions automatically, detects selected incorrect movement patterns, analyses tension and consistency, and gives feedback in a mobile app with a full workout history. The first implemented exercise is the **Biceps Curl**; the software is structured so that further exercises (Shoulder Press, Chest Press, Seated Row, Triceps Extension) can be added later.

1.2 System functionality

- Two MPU6050 IMUs and an S-type load cell (through an HX711) are sampled by an ESP32 at 50 Hz.
- The ESP32 streams binary sensor packets to the phone over Bluetooth Low Energy (BLE).
- The React Native app decodes, validates and batches the data and uploads it to the FastAPI backend about once per second.
- The backend runs the Python ML pipeline: filtering, elbow-angle estimation, repetition counting and Random Forest quality classification.
- Results are stored in MongoDB Atlas and returned to the app, which shows live feedback and the final workout report.

1.3 Architecture

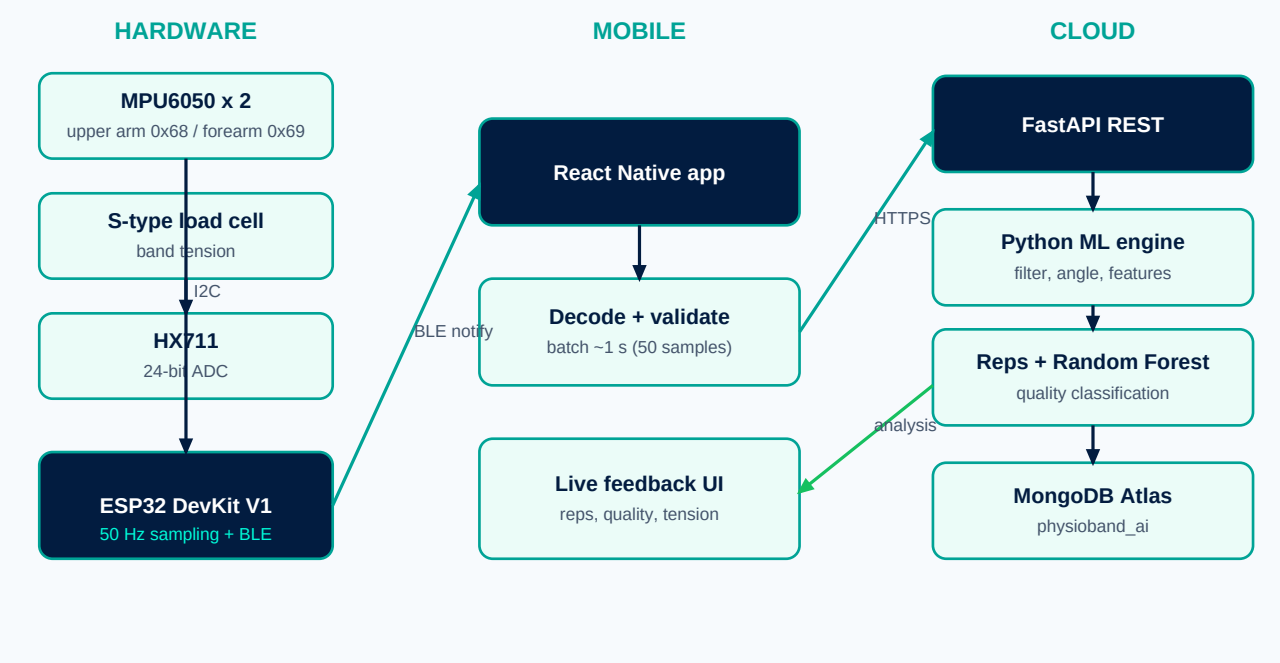


Figure 1 - PhysioBand AI architecture and data flow (the only data path in the system)

1.4 Responsibilities

Layer	Technology	Responsibility
Hardware / firmware	ESP32, Arduino C++	Sampling, timestamps, sensor validation flags, calibration, BLE
Mobile app	Expo, React Native, TypeScript	BLE connection, decoding, batching, live UI, history, profile

Layer	Technology	Responsibility
Backend	FastAPI, Pydantic, Motor	Authentication, devices, workouts, sensor storage, analytics
ML engine	NumPy, SciPy, scikit-learn	Preprocessing, rep counting, features, classification
Database	MongoDB Atlas	Users, devices, exercises, sessions, sensor batches, feedback

1.5 Movement-quality classes (Biceps Curl)

Class	Meaning
Correct	Full range of motion, controlled tempo, upper arm steady
Incomplete	Clearly reduced range of motion
Excessively fast	Full range but very short repetition
Irregular	Jerky / stop-and-go movement or swinging the upper arm

These sensors cannot detect every possible posture error; only the four classes above are claimed.

Chapter 2 - Components Required

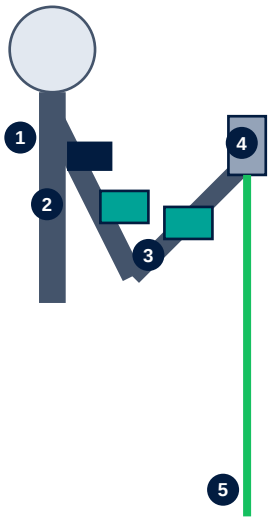
Only the approved components below are used. No additional sensors, displays, LEDs or buzzers are required. The ESP32 has Bluetooth Low Energy built in, so no separate Bluetooth module is needed.

Component	Qty	Purpose / where used	Important specifications	Cables / accessories
ESP32 DevKit V1	1	Main controller on the PCB: reads sensors, BLE	Dual-core 240 MHz, BLE 4.2, 3.3 V logic, VIN 5 V input	Micro-USB data cable
MPU6050 GY-521	2	Upper-arm and forearm motion (accel + gyro)	3-axis accel + gyro, I2C 0x68/0x69 via AD0, on-board regulator	Jumper wires, arm straps
S-type load cell	1	In-line between handle and band: band tension	Capacity >= 2 x max band force (Ch. 3.4), threaded ends	Eye bolts, carabiners rated above capacity
HX711 amplifier	1	Converts load-cell bridge signal to digital	24-bit ADC, gain 128, 10/80 SPS	Screw terminals / jumpers
Protected 18650, 3.7 V	1	Portable power	Built-in protection circuit, >= 2000 mAh recommended	18650 holder
TP4056 with protection	1	Charges the 18650 safely	1 A charging, B+/B- battery, OUT+/OUT- load (NOT a regulator)	USB charging cable
Regulated 5 V boost	1	Raises 3.0-4.2 V battery to stable 5 V for ESP32 VIN	Output set/verified at 5.0 V, >= 500 mA	-
Resistance band	1	Exercise resistance	Known resistance level; inspect for wear	Handle
Jumper wires	1 set	All electrical connections	Male/female, keep I2C runs short	-
General-purpose PCB	1	Holds ESP32, HX711, TP4056, boost converter	Perforated board, headers	Solder, headers
18650 battery holder	1	Mounts the cell	Correct polarity marking	-
Straps & mounting	1 set	Fix sensors / PCB to arm, mount load cell	Elastic hook-and-loop straps, enclosure	Eye bolts, lock nuts, carabiners

Why every component is required

- **Two IMUs**: the elbow angle is computed from the difference between upper-arm and forearm orientation; one IMU alone cannot separate elbow flexion from whole-arm swinging.
- **Load cell + HX711**: measures the actual band tension, used for tension analysis and consistency features.
- **ESP32**: fixed-rate sampling, timestamps and BLE in one low-cost board.
- **18650 + TP4056 + boost**: wireless operation during exercise; TP4056 charges, the boost converter regulates.
- **PCB, holder, wires, straps**: robust, wearable assembly.

Chapter 3 - Hardware Assembly



1 PCB (ESP32, HX711, power) on upper-arm strap or waist belt

2 Upper-arm MPU6050 (0x68): lateral side, mid upper arm

3 Forearm MPU6050 (0x69): lateral side, mid forearm

BOTH: chip facing out, X arrow pointing to the hand

4 S-type load cell in-line: handle - cell - band

5 Resistance band anchored under the foot

Load path in pure tension, along the cell axis

Figure 2 - Sensor, PCB and load-cell placement for the Biceps Curl

3.1 Upper-arm MPU6050 (address 0x68)

- Lateral (outer) side of the upper arm, halfway between shoulder and elbow, on a flat muscular area.
- Chip side facing away from the body; the printed X arrow points down the arm towards the hand when the arm hangs straight.
- Strap firmly so it cannot slide or rotate.

3.2 Forearm MPU6050 (address 0x69)

- Lateral side of the forearm, halfway between elbow and wrist.
- **Exactly the same orientation as the upper-arm sensor** (chip outward, X arrow towards the hand). The ML pipeline computes the elbow angle as the angle between the two gravity directions, which is only valid with identical orientation.

3.3 ESP32, PCB and battery

- Solder the ESP32, HX711, TP4056 and boost converter onto the general-purpose PCB using headers.
- Mount the PCB and 18650 holder in an enclosure on the upper-arm strap (above the IMU) or on a waist belt, so cables to both IMUs and the load cell stay short.
- Keep the TP4056 USB socket accessible for charging and the ESP32 USB socket accessible for programming.

3.4 S-type load cell selection and safe installation

1. **Measure the expected maximum tension**: attach the strongest band to a luggage scale and stretch it to the length reached at the top of a curl.
2. **Choose a capacity of at least 2 x that value** (safety margin for jerks). Example: 12 kg (~118 N) measured -> 30 kg or 50 kg cell. Avoid a much larger capacity (weak, noisy signal).

3. Every part in the load path (eye bolts, threads, carabiners, band attachment) must be rated above the cell capacity.
4. Thread eye bolts fully into both ends (thread size from the datasheet, e.g. M6/M8/M10) and lock them with nuts.
5. Connect: **handle - carabiner - load cell - carabiner - band**. The cell must hang freely in line with the band (pure tension, no bending or side load).
6. Strain-relieve the load-cell cable so the band never pulls on the wires.
7. Inspect bolts, threads and band before every session; never exceed the rated capacity.

SAFETY: The load cell and its mounting hardware must safely withstand the applied tensile force. Do not assume a capacity without measuring the band.

Chapter 4 - Circuit Connections

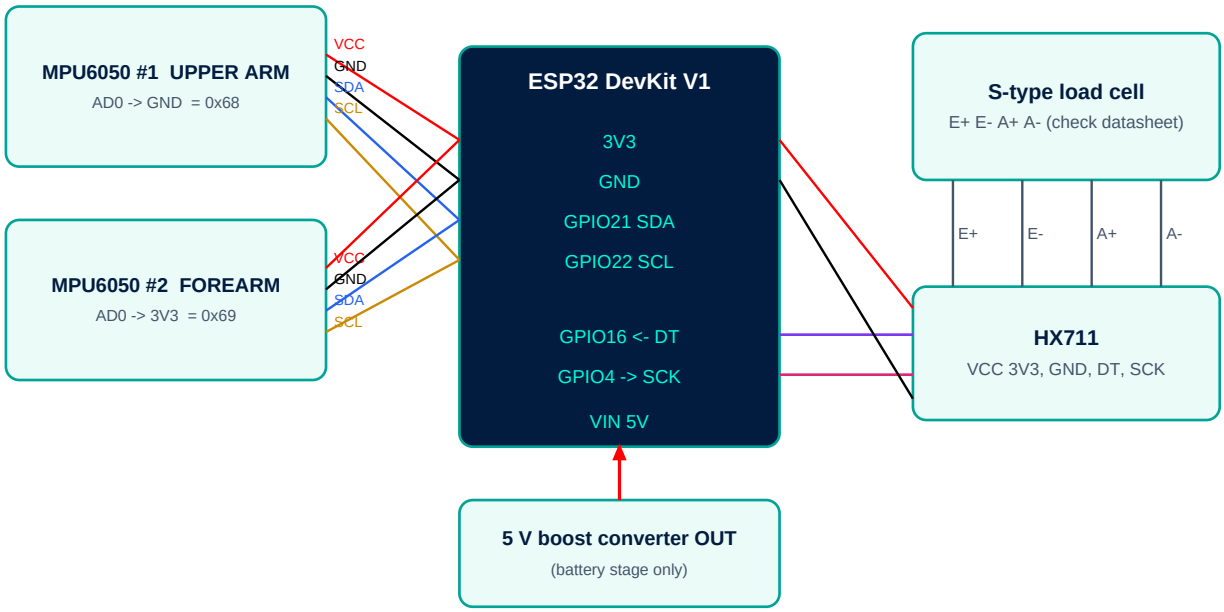


Figure 3 - Signal wiring (all sensors on 3V3, shared I2C bus)

4.1 Two MPU6050 sensors on one I2C bus

GY-521 pin	Upper-arm sensor	Forearm sensor	ESP32
VCC	VCC	VCC	3V3
GND	GND	GND	GND
SDA	SDA	SDA	GPIO 21
SCL	SCL	SCL	GPIO 22
AD0	GND -> address 0x68	3V3 -> address 0x69	-
XDA / XCL / INT	not connected	not connected	-

Both modules share SDA and SCL (a bus). Each I2C device must have a unique address; the AD0 pin selects 0x68 (LOW) or 0x69 (HIGH). Never leave AD0 unconnected. Keep the forearm cable short (< 50 cm) and twist SDA/SCL with GND wires. If communication errors occur, set `I2C_CLOCK_HZ` to `100000UL` in config.h.

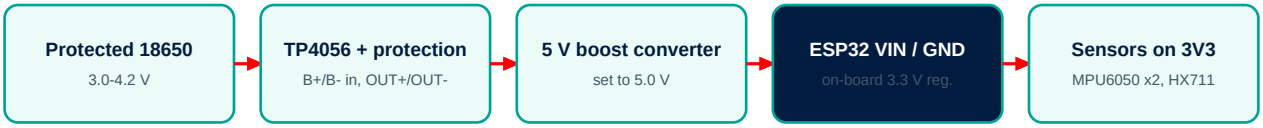
4.2 HX711 and S-type load cell

Load-cell wire	HX711	Function
Excitation +	E+	Bridge supply +
Excitation -	E-	Bridge supply -
Signal +	A+	Output + (channel A, gain 128)
Signal -	A-	Output -
Shield (if any)	GND	Optional noise shield

HX711 pin	ESP32	Note
VCC (and VDD)	3V3	DT logic level = supply; 3.3 V keeps it ESP32-safe
GND	GND	
DT / DOUT	GPIO 16	PIN_HX711_DT in config.h (configurable)
SCK / PD_SCK	GPIO 4	PIN_HX711_SCK in config.h (configurable)

IMPORTANT: Load-cell wire colours are NOT standardised. Red/black/green/white is common but not guaranteed. Use the manufacturer's datasheet. Without one: measure the resistance of all six wire pairs - the excitation pair and the signal pair read the full bridge resistance (excitation usually slightly higher, e.g. ~400 vs ~350 ohm), the other four pairs read about 3/4 of it. If force reads negative when pulling, swap A+ and A-.

4.3 Power supply



First tests: power the ESP32 from USB only. Never feed battery/TP4056 OUT directly to the ESP32.

Figure 4 - Battery power chain (only after USB tests pass)

From	To
18650 holder + / -	TP4056 B+ / B-
TP4056 OUT+ / OUT-	Boost converter IN+ / IN-
Boost OUT+ (5.0 V)	ESP32 VIN (5V)
Boost OUT-	ESP32 GND
ESP32 3V3 / GND	Both MPU6050 and HX711

- **TP4056 is a charging module, not a voltage regulator**; its OUT pins carry raw battery voltage (3.0-4.2 V).
- Never connect the battery or TP4056 OUT directly to the ESP32 5V/VIN or 3V3 pins. Always use the boost converter.
- Measure the boost converter output (5.0 V) with a multimeter before connecting the ESP32.
- Use a protected cell and a TP4056 module with protection IC; check that the cell accepts ~1 A charge current.
- Do not connect USB and battery power simultaneously unless your board schematic confirms diode isolation.
- There is no switch in the approved list: switch off by disconnecting the boost output jumper or removing the cell.

4.4 Safety checklist

#	Check before powering
1	First test on USB power only, battery circuit disconnected
2	All grounds common

#	Check before powering
3	Sensors powered from 3V3, never 5 V
4	Boost output measured at 5.0 V
5	Battery polarity correct, no exposed contacts
6	TP4056 OUT never wired directly to the ESP32
7	Load-path hardware rated above band force
8	Cables strain-relieved

Chapter 5 - Development Environment Setup (Windows)

Tool	Download	Notes
Python 3.11 / 3.12	python.org/downloads	Tick 'Add python.exe to PATH'
VS Code	code.visualstudio.com	Extensions: Python, ESLint, Prettier
Arduino IDE 2.x	arduino.cc/en/software	For the ESP32 firmware
Node.js LTS (22.x)	nodejs.org	Includes npm and npx
Git	git-scm.com	Version control / GitHub
Android Studio	developer.android.com/studio	Android SDK + platform tools for the development build
MongoDB Atlas	mongodb.com/atlas	Free cloud database (Chapter 9)

Verify the installations in **Windows PowerShell**:

```
python --version
node --version
npm --version
git --version
```

If PowerShell blocks virtual-environment activation, allow local scripts once for your user:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Get the project (clone from your GitHub repository or copy the folder):

```
cd "D:\Student Projects"
git clone https://github.com/<your-account>/physioband-ai.git PhysioBand
cd PhysioBand
```

React Native uses Expo. No global install is needed; commands run through `npx expo ...`. For the Android development build, Android Studio must install an SDK and you need USB debugging enabled on the phone (Settings -> About phone -> tap Build number 7 times -> Developer options -> USB debugging).

Chapter 6 - ESP32 Firmware Setup

1. Install **Arduino IDE 2.x**.
2. **ESP32 board package**: File -> Preferences -> Additional boards manager URLs: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`. Then Tools -> Board -> Boards Manager -> search **esp32** (Espressif) -> Install (tested 3.3.7).
3. **Library**: Sketch -> Include Library -> Manage Libraries -> install **HX711** by Bogdan Necula. (BLE, Wire, Preferences are part of the ESP32 package.)
4. Open `firmware/esp32_physioband/esp32_physioband.ino` (the .h files open as tabs).
5. Tools -> Board -> esp32 -> **ESP32 Dev Module**.
6. Connect the ESP32 by USB; Tools -> Port -> select the new **COM** port (install the CP210x or CH340 USB driver if none appears).
7. Click **Upload**. If it stays at 'Connecting...', hold the BOOT button until uploading starts.
8. Open Tools -> Serial Monitor at **115200 baud**, line ending **Newline**.

6.1 Verify the sensors

```
=== PhysioBand AI firmware 1.0.0 ===
[I2C] Devices found: 0x68 0x69
[IMU] upper-arm at 0x68 OK
[IMU] forearm at 0x69 OK
[HX711] OK, factor=21000.00 counts/kg (DEFAULT - run 'cal <kg>'), offset=...
[BLE] Advertising as 'PhysioBand-A2F0'
```

Command	Purpose
status	Sensor, calibration and BLE status
i2c	Scan the I2C bus (expect 0x68 and 0x69)
read	One sample in physical units (g, deg/s, N)
debug on / off	Print a sample every 2 s
tare	Zero the load cell (band attached but slack)
cal	Calibrate with a known hanging mass
factor	Set the HX711 factor directly
gyrocal	Gyro bias calibration (sensors still for 3 s)
csv on / off	CSV stream for ml/scripts/collect_data.py

6.2 Testing MPU6050 readings

Send `read` with the sensors lying flat: one accelerometer axis reads about +/-1.0 g, the others near 0, gyro near 0 deg/s. Rotate a sensor: the gyro values change and the 1 g moves between axes.

6.3 Testing HX711 and calibrating the load cell

1. `tare` with no load.
2. Pull the band: `read` shows the raw value and force changing.
3. Hang a known mass, e.g. 2.0 kg, freely from the cell and send `cal 2.0`. The factor is saved in flash.
4. Check: `read` should show about 19.6 N (2.0 x 9.81).

5. Place both IMUs still and send `gyrocal`.

IMPORTANT: HX711_DEFAULT_CALIBRATION_FACTOR in config.h is only a placeholder. The real factor depends on your specific load cell and MUST be measured with 'cal '.

6.4 Testing BLE

Install a generic BLE scanner app (for example nRF Connect) on the phone. A device named **PhysioBand-XXXX** with service `7b1e0001-5c3a-4d8e-9f21-6a0b3c4d5e6f` must appear. Then test with the PhysioBand app (Chapter 7).

Chapter 7 - Frontend (Mobile App) Setup

IMPORTANT: react-native-ble-plx is a native module. It does NOT work in the standard Expo Go app. You must build and install an Expo development build (npx expo run:android or EAS Build). Expo Go / web can only be used to preview screens; BLE then shows 'unavailable'.

1. Open the folder in a terminal: `cd frontend`
2. Install dependencies: `npm install` (BLE, navigation, SVG charts, secure storage are already listed in package.json).
3. Create `.env` from `.env.example` and set the backend URL (use your PC's IPv4 from `ipconfig`, not localhost).
4. Connect the Android phone by USB with USB debugging enabled (check with `adb devices`).
5. Build and install the development build: `npx expo run:android` (first build takes several minutes).
6. Later runs: `npx expo start --dev-client` and open the installed PhysioBand AI app.

```
cd frontend
npm install
copy .env.example .env
notepad .env        # EXPO_PUBLIC_API_URL=http://192.168.1.20:8000/api/v1
npx expo run:android
```

Alternative without Android Studio: `npx eas-cli@latest build --profile development --platform android` builds in the cloud and gives an installable APK.

7.1 Bluetooth permissions

On first scan Android asks for **Nearby devices** (Android 12+) or **Location** (Android 11 and below). Allow it. If denied: Settings -> Apps -> PhysioBand AI -> Permissions. Bluetooth must be switched on.

7.2 Scanning and receiving sensor data

1. Power the ESP32 (USB first).
2. In the app: Device tab -> **Scan for devices** -> tap **PhysioBand-XXXX** -> Connect.
3. The Device screen shows 'Receiving data', ~50 Hz, and the status of upper-arm IMU, forearm IMU, load cell and calibration, plus a live tension trace.
4. The device is registered automatically with the backend on each connection.
5. Tare and gyro calibration can also be started from this screen.

7.3 App screens

Screen	Content
Splash	Logo and loading animation while the session is restored
Login / Register	Email + password, validation, backend auth
Dashboard	Today's reps, correct / incorrect, quality ring, week chart, recent workouts, Start Workout
Device connection	Scan, connect / disconnect, sensor availability, tension, calibration
Exercise selection	Biceps Curl (active); other exercises shown as Coming soon
Live workout	Rep counter, per-rep feedback, counts per class, arm status, band tension, cloud/BLE status

Screen	Content
Workout result	Totals, breakdown, tension, duration, per-rep chart, suggestions
History	Quality trend, reps per day, paginated sessions
Profile & settings	Account, password, devices, AI model info, logout

Chapter 8 - Backend Setup

```
cd backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
copy .env.example .env
notepad .env
```

Variable	Value
MONGODB_URI	Atlas connection string (Chapter 9)
MONGODB_DB_NAME	physioband_ai
JWT_SECRET_KEY	long random secret (see command below)
ML_MODEL_PATH	../ml/saved_models/biceps_curl_quality.pkl
CORS_ORIGINS	* (development)

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

Start the API (0.0.0.0 makes it reachable from the phone on the same Wi-Fi):

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

- Swagger UI: `http://localhost:8000/docs` - try every endpoint ('Authorize' with the token from login).
- Health: `http://localhost:8000/health` - shows database connection and model status.
- Allow Python through the Windows Firewall on **private** networks when prompted, otherwise the phone cannot connect.
- Run tests: `python -m pytest -q` (24 tests, in-memory database).

8.1 Main endpoints

Method	Endpoint	Purpose
POST	/api/v1/auth/register, /auth/login	Account and JWT token
GET	/api/v1/auth/me	Current user
POST/GET	/api/v1/devices/register, /devices	ESP32 registration
GET	/api/v1/exercises	Exercise catalogue
POST	/api/v1/workouts/start	Start a session
POST	/api/v1/workouts/{id}/sensor-data	Upload batch, live analysis
POST	/api/v1/workouts/{id}/analyze, /complete	Full analysis, finish session
GET	/api/v1/workouts, /workouts/{id}	History, detail
POST/GET	/api/v1/ml/predict, /ml/model-info	Stateless prediction, model status
GET	/api/v1/dashboard/summary, /performance	Analytics

Chapter 9 - MongoDB Atlas Setup

1. Create an account at **mongodb.com/atlas**.
2. Create a project, e.g. **PhysioBand AI**.
3. Create a cluster: choose the **free tier (M0)** if available and a region close to you.
4. **Database Access** -> Add New Database User -> username + strong password, role 'Read and write to any database'.
5. **Network Access** -> Add IP Address -> 'Add current IP address' (for demos you may add 0.0.0.0/0 temporarily; remove it afterwards).
6. **Database -> Connect -> Drivers -> Python**: copy the connection string `mongodb+srv://<user>:<password>@<cluster>/?retryWrites=true&w=majority`.
7. Paste it into `backend/.env` as MONGODB_URI (URL-encode special characters in the password, e.g. @ -> %40).
8. Start FastAPI; the log shows *Connected to MongoDB database 'physioband_ai'* and /health shows connected: true.
9. The backend creates the six collections, their indexes and the exercise catalogue automatically on first start.
10. After a workout, open **Browse Collections** and check the saved data (below).

Collection	Contents	Key index
users	name, email, password hash, profile	unique email
devices	device_id, name, firmware, user	unique (user_id, device_id)
exercises	catalogue, is_active flag	_id slug
workout_sessions	times, reps, quality, tension, status	(user_id, start_time)
sensor_records	one document per ~1 s batch (arrays)	unique (session_id, batch_seq)
workout_feedback	one document per repetition	unique (session_id, repetition_number)

9.1 Verifying saved workout data

- workout_sessions: filter `{ "status": "completed" }`, sort `{ "start_time": -1 }` - totals must match the app.
- workout_feedback: filter `{ "session_id": "<session id>" }` - one document per rep.
- sensor_records: same filter - sample_count values add up to the session's sample_count.

SAFETY: Keep credentials secret: they live only in backend/.env, which is listed in .gitignore. Never hard-code passwords in source files, never commit .env, and rotate the database password if it was ever shared.

Chapter 10 - ML Model Setup

```
cd ml
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python -m pytest -q
```

10.1 Synthetic demonstration data vs. real collected data

Synthetic data (`scripts/generate_synthetic_data.py`) is simulated from a simple physical model. It only proves that the software pipeline works end to end. The included demo model was trained on 10 simulated participants and scores 100 % on 3 held-out simulated participants - this number is **meaningless for real use** because the synthetic classes were generated to be separable. **Real data** recorded from volunteers with the actual hardware is required for a meaningful model; report only metrics from real held-out volunteers.

10.2 Collecting and labelling real samples

1. Mount the sensors (Chapter 3), `tare`, `gyrocal`, close the Arduino Serial Monitor.
2. For each volunteer record one set per label (8-12 reps): correct, incomplete, too_fast, irregular.
3. Run: `python scripts\collect_data.py --port COM5 --participant P01 --label correct --reps 10`, press Enter, perform the set, Ctrl+C.
4. Repeat for every label and volunteer (at least 3 volunteers, ideally 8 or more). Files go to data/raw/<participant>/.
5. Validate: `python scripts\validate_dataset.py data\raw` (rep count per set, sensor issues).

CSV columns: participant_id, session_id, exercise_id, label, is_synthetic, timestamp_ms, ua_ax..ua_gz, fa_ax..fa_gz, force_n.

10.3 Features, training, evaluation

```
python scripts\extract_features.py data\raw --out data\processed\features_collected.csv
python scripts\train_model.py --data data\raw
python scripts\evaluate_model.py --data data\raw\P09
python scripts\predict.py data\sample\sample_biceps_curl_synthetic.csv
```

- Pipeline: validation -> resampling to 50 Hz -> calibration -> 5 Hz Butterworth low-pass -> complementary-filter gravity tracking -> elbow angle -> peak/cycle rep counting -> 31 features -> Random Forest.
- Split is **participant-wise** (no volunteer in both train and test).
- Reports: accuracy, precision, recall, F1 per class, confusion matrix PNG, feature importance PNG in ml/reports/.
- The model is saved to ml/saved_models/biceps_curl_quality.pkl, which FastAPI loads at start-up. Restart the backend after retraining.
- Check: `GET /api/v1/ml/model-info` shows algorithm, version, dataset_type (synthetic / collected) and validated: false.

Chapter 11 - Connecting Everything

ESP32 -> BLE -> React Native -> FastAPI -> Python ML -> MongoDB Atlas -> mobile feedback

11.1 BLE UUID configuration

Item	Value
Device name	PhysioBand-XXXX
Service	7b1e0001-5c3a-4d8e-9f21-6a0b3c4d5e6f
SENSOR_DATA (notify)	7b1e0002-5c3a-4d8e-9f21-6a0b3c4d5e6f
CONTROL (write)	7b1e0003-... : 0x01 start, 0x02 stop, 0x03 tare, 0x04 gyro cal
DEVICE_INFO (read)	7b1e0004-... : JSON device id, firmware, protocol, sample rate
MTU	App requests 185; packet = 8 + 33 x N bytes (N <= 4)

The same values are defined in `firmware/esp32_physioband/config.h` and `frontend/src/constants/ble.ts`. The app scans, connects, requests the MTU, discovers services, reads DEVICE_INFO, subscribes to SENSOR_DATA notifications and writes 0x01 to CONTROL.

11.2 Data packet format (protocol v1, little-endian)

Offset	Size	Field
0	1	protocol version = 1
1	1	sample count N
2	2	packet sequence
4	4	device id
8+33i	4	timestamp ms
+4	12	upper arm ax ay az (mg), gx gy gz (0.1 deg/s) int16
+16	12	forearm (same layout)
+28	4	force int32 (0.01 N)
+32	1	status bits: 0 UA ok, 1 FA ok, 2 load cell ok, 3 LC calibrated, 4 gyro calibrated

Example decoded sample (the app turns each packet into this structure):

```
{
 "timestamp_ms": 3997,
 "ua": [-0.9966, 0.0165, 0.005, -0.0026, 0.1004, 0.6697],
 "fa": [-0.8275, 0.005, 0.5428, -0.785, 88.5373, -0.3532],
 "force_n": 30.3138,
 "status": 15
}
```

11.3 API request (app -> backend, every ~1 s)

```
POST /api/v1/workouts/<session_id>/sensor-data
Authorization: Bearer <token>
{
 "batch_seq": 4,
 "device_id": "PB-3C61A2F0",
 "protocol_version": 1,
 "samples": [
  {
   "timestamp_ms": 3997,
   "ua": [-0.9966, 0.0165, 0.005, -0.0026, 0.1004, 0.6697],
```

```
    "fa": [-0.8275, 0.005, 0.5428, -0.785, 88.5373, -0.3532],
    "force_n": 30.3138,
    "status": 15
   },
   "... 49 more samples ..."
  ],
  "analyze": true
}
```


11.4 ML prediction response (backend -> app)


```
{
 "session_id": "6aeac003ba18737a91c7a777",
 "batch_seq": 4,
 "accepted": true,
 "duplicate": false,
 "total_samples": 250,
 "analysis": {
  "mode": "live",
  "total_reps": 1,
  "correct_reps": 1,
  "incorrect_reps": 0,
  "quality_score": 100.0,
  "avg_tension_n": 52.79,
  "new_reps": [
   {
    "number": 1,
    "start_timestamp_ms": 1900.0,
    "end_timestamp_ms": 4100.0,
    "duration_s": 2.2,
    "rom_deg": 106.6,
    "avg_force_n": 52.79,
    "peak_force_n": 68.56,
    "label": "correct",
    "label_display": "Correct Movement",
    "confidence": 0.9582,
    "probabilities": {
     "correct": 0.9582,
     "incomplete": 0.0351,
     "irregular": 0.005,
     "too_fast": 0.0017
    },
    "feedback": "Good repetition: full range of motion with a controlled tempo."
   }
  ],
  "live": {
   "movement_phase": "resting",
   "upper_arm_status": "stable",
   "forearm_status": "active",
   "current_angle_deg": 13.2,
   "current_force_n": 22.61,
   "upper_arm_swing_deg": 0.3
  },
  "model": {
   "available": true,
   "algorithm": "RandomForestClassifier",
   "dataset_type": "synthetic",
   "validated": false
  }
 }
}
```


These values were produced by the real code with a synthetic stream and the demo model.

11.5 MongoDB storage

- Each batch -> one sensor_records document (arrays of the 50 samples).
- Each newly completed rep -> one workout_feedback document.

- Session totals (reps, quality, tension) -> workout_sessions, finalised by /complete.

11.6 Error and reconnection handling

Situation	Behaviour
BLE drop	App retries 5 x every 2.5 s; firmware re-advertises; banner in Live Workout
No internet	Batches queued and re-sent in order; app shows 'Cloud AI analysis unavailable' - no fabricated predictions
Retried upload	Same batch_seq ignored (idempotent)
App closed mid-workout	Session closed as interrupted when the next workout starts
Sensor fault	Status bit clears -> values sent as null -> pipeline interpolates short gaps
Database down	/health reports it, API returns 503

Chapter 12 - Testing and Troubleshooting

Problem	How to verify	Fix
MPU6050 not detected	'i2c' misses 0x68/0x69	Check 3V3, GND, SDA 21, SCL 22; re-seat wires
I2C address conflict	Only one address found	Upper-arm AD0 to GND, forearm AD0 to 3V3
Incorrect orientation	Elbow angle not ~0 deg with straight arm	Both sensors: chip out, X arrow to hand
HX711 shows zero	'HX711 NOT RESPONDING'	DT 16, SCK 4, 3V3; check E+/E-/A+/A-
Calibration problems	2 kg not ~19.6 N	tare unloaded, hang mass freely, cal 2.0
BLE not discovered	Scanner app finds nothing	Firmware advertising? Development build (not Expo Go)? Permission? Distance?
BLE disconnection	'Reconnecting' banner	Stay close; auto-retry; avoid USB + battery together
Permission issues	'Permission needed'	Allow Nearby devices / Location in Android settings
FastAPI connection error	'Cannot reach the server'	Same Wi-Fi, PC IPv4 in .env, --host 0.0.0.0, firewall
MongoDB Atlas error	/health connected: false	URI, password encoding, Network Access IP, cluster running
ML model not loading	/health model.available false	Train model, check ML_MODEL_PATH, scikit-learn 1.6.1
Invalid sensor packets	Invalid packets counter / 422	Reconnect for MTU; ua/fa need 6 values
Incorrect rep count	Count differs	Curl > 25 deg, check IMU status, tune thresholds with real data

12.1 Automated tests

```
cd backend;  .\.venv\Scripts\python.exe -m pytest -q        # 24 tests
cd ..\ml;   python -m pytest -q                               # 18 tests
cd ..\frontend; npx tsc --noEmit; npx expo-doctor
```

12.2 End-to-end test without hardware

```
backend\.venv\Scripts\python.exe scripts\simulate_device.py --api http://localhost:8000/api/v1
```

The simulator logs in, streams a synthetic workout in 1-second batches, prints per-rep feedback and completes the session.

NOTE: Tests requiring the physical hardware cannot be completed in software: real I2C detection, orientation, HX711 calibration, BLE with a phone, battery operation and real-world accuracy. Record their results yourself during integration.

Chapter 13 - Complete Project Execution (Demonstration Day)

1. **Connect hardware**: sensors strapped (same orientation), load cell in-line, PCB on strap. Power by USB (or the verified battery circuit).
2. **Upload ESP32 firmware** (if not already); Serial Monitor -> `status` shows both IMUs, HX711 OK, calibrated. Close the Serial Monitor.
3. **Start MongoDB Atlas**: log in, check the cluster is running (resume if paused) and your current IP is in Network Access.
4. **Start FastAPI**: `cd backend; .\.venv\Scripts\Activate.ps1; uvicorn app.main:app --host 0.0.0.0 --port 8000`
5. **Verify the ML model**: open http://localhost:8000/health -> database connected: true, model available: true.
6. **Start the app**: `cd frontend; npx expo start --dev-client`, open PhysioBand AI on the phone, log in.
7. **Connect the ESP32**: Device tab -> Scan -> PhysioBand-XXXX -> check 'Receiving data' and all sensors OK; tare the load cell.
8. **Select Biceps Curl**: Dashboard -> Start Workout -> Biceps Curl.
9. **Start a workout**: press Start; timer runs and status chips show 'Sensors 50 Hz' and 'Cloud AI live'.
10. **Perform repetitions**: a few correct curls, then one incomplete, one fast and one jerky curl to demonstrate feedback.
11. **View AI analysis**: rep counter, per-rep feedback card, counts per class, arm status and band tension update live.
12. **Complete the workout**: press Finish workout -> result screen with totals, tension, per-rep chart and suggestions.
13. **Verify in MongoDB Atlas**: Browse Collections -> workout_sessions (newest), workout_feedback and sensor_records for that session id; the app's History tab shows the session.

TIP: Explain to examiners that the shipped classifier is a demo trained on synthetic data. Show the real-data workflow (collect_data -> train_model) and present only metrics obtained from real held-out volunteers.